

25. Teorie grafů (pojem grafu, isomorfismus, souvislost, algoritmy nejkratší cesty a minimální kostry)

25. Teorie grafů

SZZ okruh 25 (FIT BUT). Grafy jako speciální binární relace + klíčové algoritmy.

Shrnutí

Základní pojmy

- Graf $G = (V, E)$ — vrcholy a hrany (dvoupřvkové podmnožiny V); **stupeň** vrcholu, orientovaný graf (vstupní/výstupní stupeň).
- **Sled** (opakování dovoleno) $\neq$ **tah** (neopakuje hrany) $\neq$ **cesta** (neopakuje vrcholy/hrany).
- Typy: kružnice C_n , cesta P_n , úplný K_n , úplný bipartitní $K_{m,n}$, hvězda; **Eulerovský** graf (souvislý, všechny vrcholy sudého stupně $\neq$ jeden uzavřený tah).
- **Izomorfismus** = bijekce vrcholů zachovávající hrany; nutné (ne postačující): stejný počet vrcholů, hran a posloupnost stupňů.

Souvislost a stromy

- **Souvislý** = mezi každými dvěma vrcholy existuje sled; relace „spojen sledem“ je ekvivalence $\neq$ **komponenty souvislosti**.
- Orientované: **slabá** $\times$ **silná** souvislost.
- **Strom** = souvislý graf bez kružnic (acyklický); mezi dvěma vrcholy právě jedna cesta. **Kostra** = stromový podgraf propojující všechny vrcholy.

Grafové algoritmy

- **BFS** (fronta) — nejkratší cesta v **neváženém** grafu; **DFS** (zásobník).
- **Dijkstra** (prioritní fronta) — nejkratší cesta v **kladně váženém** grafu; $O((V + E) \cdot \log V)$.
- **Jarník/Prim** a **Kruskal** — **minimální kostra** váženého grafu (greedy).

Grafy jako relace viz [[okruhy/16-mnoziny-relace-zobrazeni]]; prohledávání viz [[okruhy/26-reseni-uloh-prohledavani]]; Dijkstra v [[okruhy/44-smerovani-zabezpeceni-siti | Link-State směrování]].

Související syntéza

- [[synthesis/slozitost-napric-algoritmy | Asymptotická složitost napříč algoritmy]] — syntéza
- [[synthesis/relace-napric-obory | Relace $\times$ graf $\times$ tabulka]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Základní pojmy □ [[#Základní pojmy]] - *Formální definice grafu?* □ $G = (V, E)$, V vrcholy, E hrany (dvojprvkové podmnožiny V). - *Sled / tah / cesta?* □ Sled (vše se opakuje), tah (neopakuje hrany), cesta (neopakuje vrcholy ani hrany). - *Izomorfismus?* □ Bijekce vrcholů zachovávající hrany; nutné stejné počty vrcholů/hran a posloupnost stupňů.

Souvislost a stromy □ [[#Souvislost a stromy]] - *Souvislost?* □ Mezi každými dvěma vrcholy existuje sled; komponenty = třídy ekvivalence. - *Strom / kostra?* □ Strom = souvislý acyklický graf; kostra = stromový podgraf přes všechny vrcholy.

Algoritmy □ [[#Grafové algoritmy]] - *Nejkratší cesta?* □ BFS (nevážený), **Dijkstra** (kladně vážený, prioritní fronta). - *Minimální kostra?* □ **Kruskal** (přidává nejlevnější hranu bez kružnice) a **Jarník/Prim** (roste z vrcholu); oba greedy. - *Reprezentace grafu?* □ Matice sousednosti / seznam sousedů.

Plné znění (ke studiu)

Neformálně se graf skládá z vrcholů a hran, které tyto vrcholy spojují. Jedná se o speciální případy binárních relací. **Formálně** je graf (jednoduchý neorientovaný) uspořádaná dvojice $G = (V, E)$

- **V** (vertices) je množina vrcholů,
- **E** (edges) je množina hran – množina vybraných **dvouprvkových podmnožin množiny vrcholů** (znázorňují spojení mezi vrcholy).

Hranu mezi vrcholy **u** a **v** píšeme jako **{u, v}**, nebo zkráceně **uv**.

- **sousední vrcholy**: vrcholy spojené hranou.

Hrana **uv** vychází z vrcholů **u** a **v**.

Množiny grafu G odkazujeme:

- **V(G)**: množina **vrcholů** grafu G ,
- **E(G)**: množina **hran** grafu G .

Grafy zadáváme **neformálně** graficky:

nebo formálně výčtem vrcholů a hran: [[media/szz-25/media/image16.png]]

[[media/szz-25/media/image5.png]]

Stupeň vrcholu

Stupněm vrcholu **x** v grafu G rozumíme počet hran vycházejících z vrcholu **x** (hrana vychází z obou konců současně). Stupeň vrcholu **x** v grafu G značíme **dG(x)**. Orientované grafy pak mají **vstupní** stupeň vrcholů a **výstupní** stupeň vrcholů v závislosti na tom, jestli hrana do vrcholu vstupuje, nebo z něj vystupuje. Graf je **d-regulární**, pokud všechny jeho vrcholy mají stejný stupeň **d**. **Nejvyšší stupeň** grafu G značíme $\Delta(G)$. **Nejnižší stupeň** grafu G značíme $\delta(G)$.

Základní pojmy

Sled

Sled je procházka po hranách grafu z **u** do **v**, která může obsahovat cykly. **Hrany i vrcholy** se mohou opakovat (na rozdíl od kružnice). Sled v grafu je **posloupnost vrcholů** taková, že mezi každými dvěma po sobě jdoucími vrcholy je hrana.

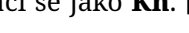
Tah

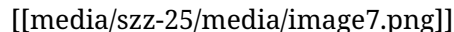
Je sled v grafu, ve kterém se **neopakují hrany** (vrcholy se opakovat mohou). **Uzavřený tah** je tah, který končí ve vrcholu, ve kterém **začal** (jinak je neuzavřený). Graf G lze nakreslit **jedním uzavřeným tahem**, právě když G je **souvislý** a všechny jeho vrcholy jsou **sudého stupně**. Viz *Problém sedmi mostů města Královce*.

Cesta

Je sled v grafu, ve kterém se **neopakují ani hrany ani vrcholy**.

Typy grafů

- **Kružnice**: Kružnice délky n má $n \geq 3$ různých vrcholů spojených **do jednoho cyklu** n hranami. Značí se jako **C n** .
- **Cesta**: Cesta délky $n \geq 0$ má $n+1$ různých vrcholů spojených za sebou n hranami. Značí se jako **P n** . (Žádné vrcholy ani hrany se neopakují.) 
- **Úplný graf**: Úplný graf na $n \geq 1$ vrcholech má n různých vrcholů spojených **po všech dvojicích** - celkem **n nad 2** (výpočet n nad k ve vzorečku; konkrétní vzoreček pro počet hran ve druhém výrazu, odvozený z prvního vzorečku pro $k = 2$) hran. Značí se jako **K n** . 
- **Úplný bipartitní graf**: Úplný bipartitní graf na $m \geq 1$ a $n \geq 1$ vrcholech má $m+n$ vrcholů ve **dvou skupinách** (partitách), přičemž hranami jsou spojeny všechny **$m \cdot n$ dvojice** z různých skupin. Značí se jako **K m,n** . 





- **Hvězda**: Hvězda s $n \geq 1$ rameny je zvláštní název pro **úplný bipartitní graf**. Značí se jako **K1, n** . 

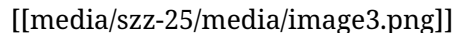


- **Eulerovský graf** - graf, který je **souvislý** a má všechny vrcholy **sudého stupně**, lze ho nakreslit **jedním uzavřeným tahem**
- **Semi-Eulerovský graf** - graf, který je **souvislý** a má právě 2 vrcholy **lichého stupně**, lze ho nakreslit **jedním otevřeným tahem**

Orientované grafy

V orientovaných grafech má každá hrana jistý směr. Formálně mají orientované grafy množinu orientovaných hran **A** , která je dána **$A \subseteq V(G) \times V(G)$** .

Podgraf



Podgrafem grafu **G** rozumíme libovolný graf **H** pro který platí:

- množina vrcholů grafu **H** je **podmnožinou** vrcholů grafu **G** : **$V(H) \subseteq V(G)$** .
- za hrany má **libovolnou podmnožinu** hran grafu **G** , které ale mají oba vrcholy ve **$V(H)$** . 

Indukovaný podgraf

Indukovaným podgrafem je podgraf $H \subseteq G$ takový, který obsahuje **všechny hrany grafu G** mezi dvojicemi vrcholů z $V(H)$. Jinak řečeno graf H vznikne smazáním části vrcholů grafu G a **pouze** hran, které vycházely z těchto vrcholů.

Typy podgrafů grafu G

[[media/szz-25/media/image8.png]]

- **Kružnice v G** je podgraf $H \subseteq G$, který je **izomorfní** nějaké **kružnici**. (kružnici délky 3 říkáme trojúhelník).
- **Indukovaná kružnice v G** je **indukovaný** podgraf $H \subseteq G$, který je **izomorfní** nějaké **kružnici**.
- **Cesta v G** je podgraf $H \subseteq G$, který je **izomorfní** nějaké **cestě**.
- **Klika v G** je podgraf $H \subseteq G$, který je **izomorfní** nějakému **úplnému grafu** (graf jehož všechny vrcholy jsou spojeny hranou se všemi zbylými).
- **Nezávislá množina X v G** je podmnožina vrcholů $X \subseteq V(G)$, mezi kterými nevedou v G **žádné hrany** (přímo tyto vrcholy spojuje hrana, spojení přes více hran existovat může).

Izomorfismus

Isomorfismus grafů G a H je **bijektivní** zobrazení $f: V(G) \rightarrow V(H)$, pro které každá dvojice $u, v \in V(G)$ je spojená hranou v grafu G **právě, když** je dvojice $f(u), f(v)$ spojená hranou v grafu H . Příklad pro grafy G a G' :

Pro izomorfní grafy G a H platí (může to platit ale i pro neizomorfní grafy): [[media/szz-25/media/image11.png]]

- G a H mají stejný počet vrcholů,
- G a H mají stejný počet hran,
- **zobrazení f** zobrazuje na sebe vrcholy **stejných stupňů**, tzn. mají stejné počty vrcholů o stejných stupních.

Postup **hledání izomorfismu** (pokud pro nějaký bod neplatí, grafy nejsou izomorfní):

1. ověříme **stejný** počet **vrcholů** u obou grafů,
2. ověříme **stejný** počet **hran** u obou grafů,
3. vytvoříme posloupnosti stupňů vrcholů pro každý graf (seřazeny od nejmenšího po největší) a ověříme, že jsou stejné.
4. zkoušíme všechny **přípustné možnosti** zobrazení izomorfismu.

Souvislost

[[media/szz-25/media/image17.png]]

[[media/szz-25/media/image10.png]]

Možnost se v grafu pohybovat z jakéhokoliv vrcholu do jakéhokoliv jiného vrcholu podél jeho hran. To znamená, že pro každé dva vrcholy $u, v \in V(G)$ existuje sled z vrcholu u do vrcholu v .

U orientovaných grafů rozlišujeme:

- **slabá souvislost** — graf je slabě souvislý, pokud jeho symetrizace (odstranění směru hran; nahrazení orientovaných hran za neorientované) je souvislý graf.

- **silná souvislost** — graf je silně souvislý, pokud pro každé dva vrcholy **u**, **v** existuje cesta z **u** do **v** i z **v** do **u**.

Relace $\sim$ na množině vrcholů $V(G)$ libovolného grafu G , je definována tak, že **u**, **v** $\in V(G)$ jsou v relaci **u** $\sim$ **v**, právě když v grafu **G** existuje **sled** začínající v **u** a končící ve **v**. Tato relace je:

- **reflexivní**: každý vrchol je spojen sám se sebou sledem délky 0.
- **symetrická**: sled z **u** do **v** lze obrátit na sled z **v** do **u** vždy u **neorientovaného** grafu.
- **tranzitivní**: dva sledy na sebe můžeme vždy **navázat v jeden**.

Relace $\sim$ je tedy relací **ekvivalence**.

Komponenty souvislosti

Jsou jednotlivé **třídy ekvivalence** grafu (graf je **souvislý**, pokud má **pouze jednu** komponentu souvislosti). Graf o třech komponentách:

Stromy

[[media/szz-25/media/image4.png]]

Strom je jednoduchý **souvislý graf T bez kružnic**. Grafy bez kružnic lze také nazývat **acyklické**. **Les** je nesouvislý graf tvořený více stromy. Stromové grafy jsou zároveň kostrou grafu. Pro stromy platí:

- pokud mají více než jeden vrchol, existuje vrchol se **stupněm 1**,
- mezi každými dvěma vrcholy vede **právě** jedna cesta.

Grafové algoritmy

Využívají nějakou paměť - zásobník, frontu, seřazené pole.

Prohledávání do šířky - Breadth First Search (BFS)

Algoritmus postupně prochází celé úrovně od počátečního vrcholu. Jako paměť využívá **frontu**. Pracuje následovně:

1. První vrchol se vloží do fronty,
2. Pokud není fronta prázdná, zpracuj vrchol na **začátku fronty**. Zpracování znamená umístění vrcholů, do kterých vede hrana ze zpracovávaného vrcholu, **do fronty**.

Algoritmus BFS lze použít pro zjištění nejkratší **vzdálenosti** mezi dvěma vrcholy spojitého **neváženého** grafu.

Prohledávání do hloubky - Depth First Search (DFS)

Algoritmus prochází nejprve do co nejvzdálenější úrovně a poté se postupně vynořuje a zase co nejvíce zanořuje. Jako paměť využívá zásobník a pracuje následovně:

1. První vrchol se vloží na zásobník,
2. Pokud není zásobník prázdný, zpracuje se vrchol na **vrcholu zásobníku**. Zpracování znamená umístění vrcholů, do kterých vede hrana ze zpracovávaného vrcholu, na **zásobník**.

Dijkstrův algoritmus

Algoritmus pro hledání **nejkratší cesty** (vzdálenosti) mezi **dvěma vrcholy u a v** v **kladně váženém** grafu. Jako úložiště používá **prioritní frontu**. Může pracovat s časovou komplexitou $O((hrany + vrcholy) \cdot \log(vrcholy))$ nebo $O(vrcholy^2)$ při použití pole. Princip algoritmu:

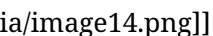
1. Všechny vrcholy až na počáteční jsou ohodnoceny **nekonečněm** (neznáme do nich cestu), počáteční vrchol je ohodnocen **0** (cesta do tohoto vrcholu je nulová). Všechny vrcholy jsou označeny za nezpracované.
2. Z nezpracovaných vrcholů vybereme ten s **nejmenší hodnotou** (na začátku prioritní fronty - při 1. iteraci to bude počáteční vrchol - vrchol **u**). Pokud je tento vrchol hledaným vrcholem (vrchol **v**), ukončíme algoritmus (pokud chceme najít nejkratší cestu ke **všem** vrcholům, **pokračujeme** např. u Link State protokolů), jinak **přepíšeme** vzdálenosti všech vrcholů (již zpracované ignorujeme), do kterých se můžeme **z vybraného vrcholu dostat hranou, přičtením ohodnocení** této hrany k hodnotě **zpracovávaného** vrcholu, pokud je tato hodnota **MENŠÍ** než aktuální ohodnocení. V tomto případě si také **zaznamenáme**, přes **jaký vrchol** vede tato nejkratší cesta.
3. Aktuálně zpracovávaný uzel označíme za zpracovaný a pokud **jsou** ještě **nezpracované vrcholy**, pokračujeme 2. bodem.
4. Zpětně **rekonstruujeme cestu** z cílového vrcholu k startovacímu vrcholu na základě **zapamatovaných** údajů v bodě 2.

Dijkstra's Algorithm - Computerphile

Jarníkův (Primův) algoritmus

Algoritmus, který hledá ve **váženém** grafu **minimální kostru** (stromový podgraf, který propojuje všechny vrcholy) - **minimum (weight) spanning tree**. Jako úložiště může použít např. prioritní frontu pro nenavštívené. Jedná se o **greedy** algoritmus, tj. jde vždy za minimem. Princip algoritmu:

1. Do úložiště ulož všechny vrcholy s ohodnocením **nekonečno** (ještě nejsou v kostře)
2. Vyber jakýkoliv uzel grafu, změň jeho ohodnocení na **0** a odstraň jej z úložiště.
3. U vrcholů (musí být v úložišti, tj. **ještě nenavštívené**), do kterých se lze z tohoto vrcholu dostat **aktualizuj** jejich **ohodnocení** hodnotou cesty k tomuto vrcholu, pokud je tato hodnota **menší než aktuální (nepřičítá se jako u Dijkstrova algoritmu)**, a zapiš vrchol, ze kterého tato **cesta vede**.
4. Vyber vrchol z úložiště, který má **nejmenší ohodnocení**, a odstraň jej z úložiště.
5. Pokud úložiště **není prázdné**, pokračuj s bodem 3.
6. Zpětně rekonstruuj použité hrany.

Prim's algorithm in 2 minutes — Review and example 



Kruskalův algoritmus

Algoritmus pro **hledání minimální kostry** ve **váženém** grafu. Pracuje na principu, že z grafu vybíráme vždy **hranu s nejmenším ohodnocením**, aby se **nevytvořila kružnice**, dokud nespojíme všechny vrcholy. Komplexita je $O(|E| \cdot \log(|V|))$.

Kruskal's Algorithm: Minimum Spanning Tree (MST)

Zdroje

- SZZ okruh 25 — studijní materiály FIT BUT (szz-25.docx). Obrázky: media/szz-25/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/24-numericke-metody| 24. Numerické metody]]
· další: [[okruhy/26-reseni-uloh-prohledavani| 26. Řešení úloh (prohledávání)]] □